

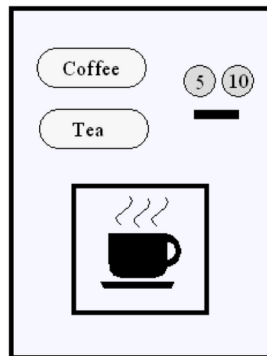
SISTEMAS DIGITALES

Modelling a Coffee Machine using Threads, Semaphores, and Message Queues

1. Behaviour of the coffee machine¹

The coffee machine in the example provides the user with a cup of coffee or a cup of hot water (for tea), given that the customer has inserted the required amount of money. The coffee machine can handle coins of the values 5 and 10, where 5 is the price for a cup of tea while a cup of coffee costs 10. The following holds:

- If a coin with the value of 10 is inserted and the `Coffee` button is pressed, the customer receives a cup of coffee (use case `MakeCoffee`).
- If a coin with the value of 10 is inserted and the `Tea` button is pressed, the customer receives a cup of hot water plus change.
- If a coin with the value of 5 is inserted and the `Coffee` button is pressed, the money is returned.
- If a coin with the value of 5 is inserted and the `Tea` button is pressed, the customer receives a cup of hot water.



2. Defining messages

The following Table shows all messages needed to implement the behaviour of the coffee machine:

To Customer	To Controller	To Hardware
sCupOfCoffee	sCoin (Integer)	sFillWater
sCupOfWater	sCoffee	sFillCoffee
sReturnChange	sTea	sHeatWater
	sWaterOK	
	sCoffeeOK	
	sWarm	

¹ Requirement taken from: IBM Corp. Telelogic Tau UML Coffee Machine Tutorial

3. Message Sequence Chart

The following Figure shows a MSC for the use case *MakeCoffee*:

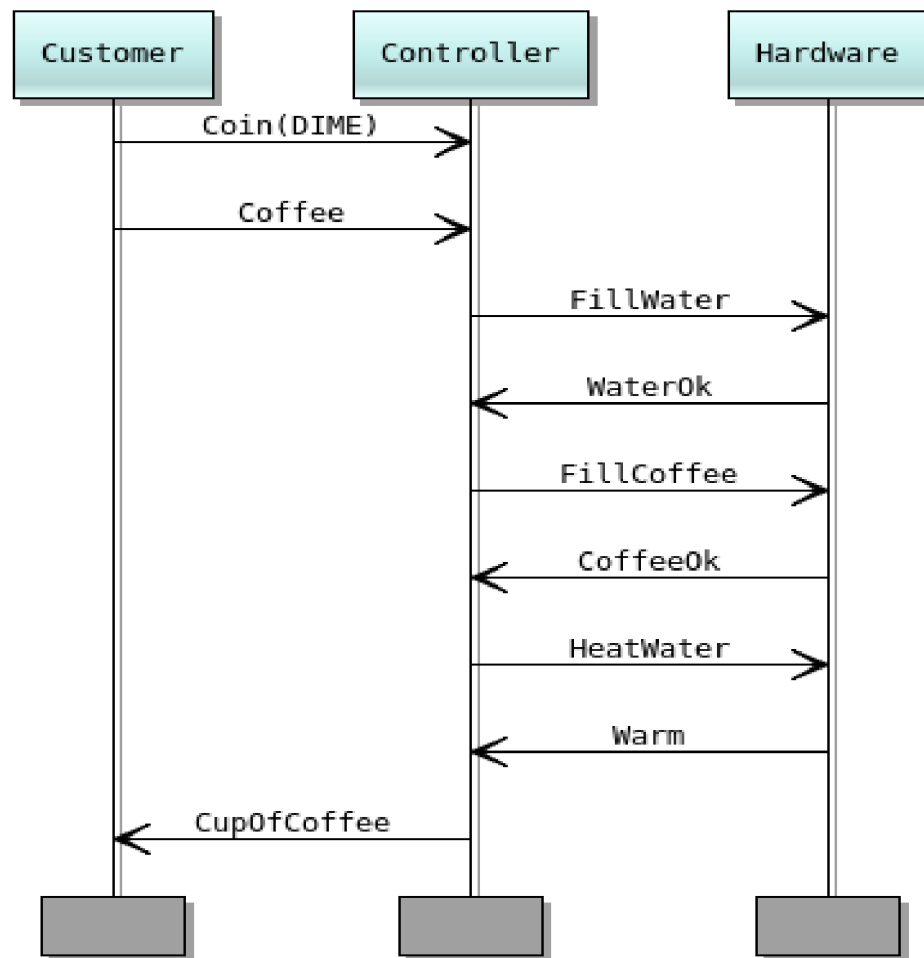


Figure 1: Requirement for Use Case *MakeCoffee*

4. Process Diagrams

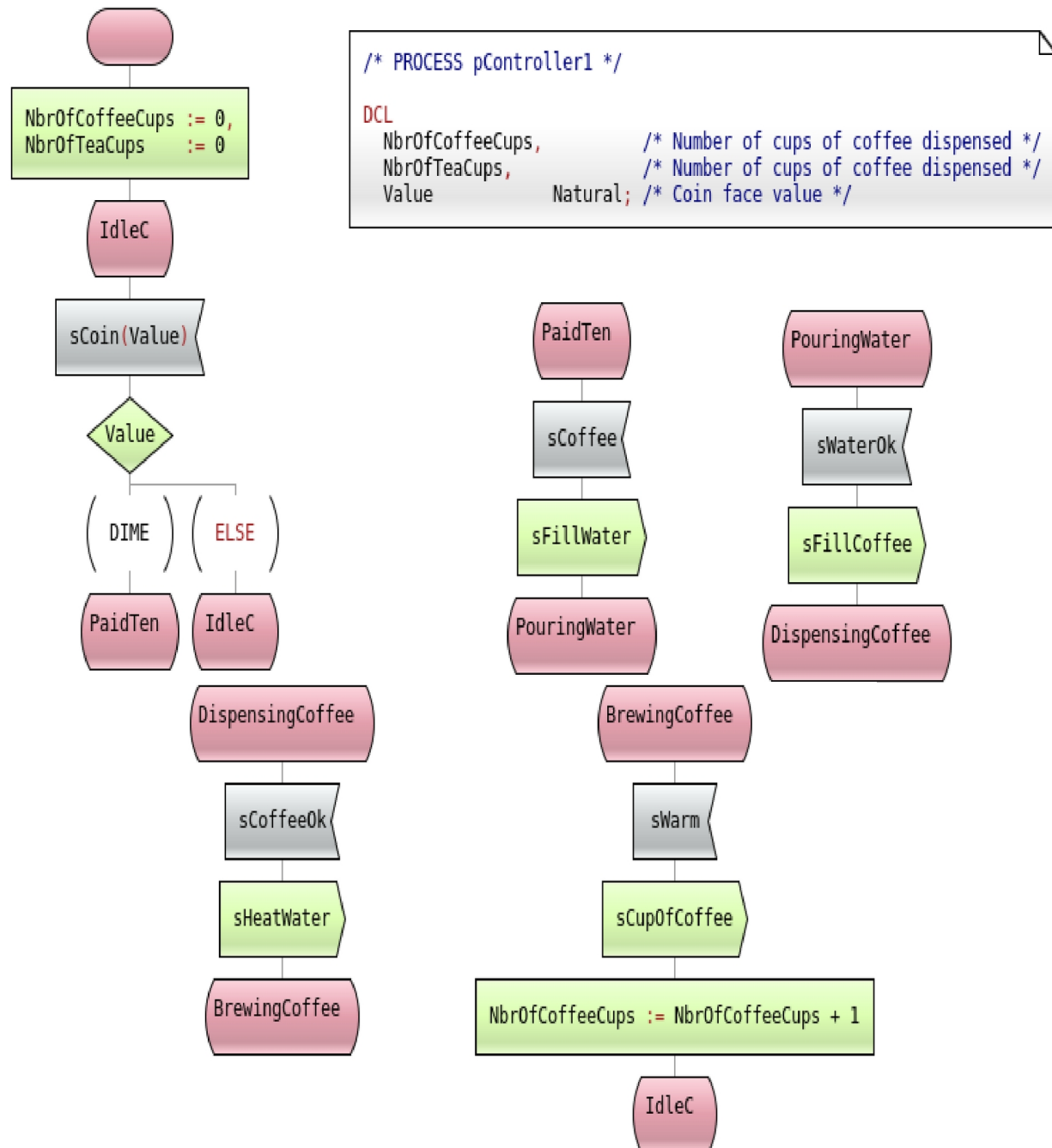
An *Extended Finite State Machine (EFSM)* diagram describes the behaviour of an SDL process. The EFSM has one or more possible states and a change of state is triggered by a message reception. The EFSM has been extended with data handling, meaning that message data and other variables can be declared and handled.

Customer Process

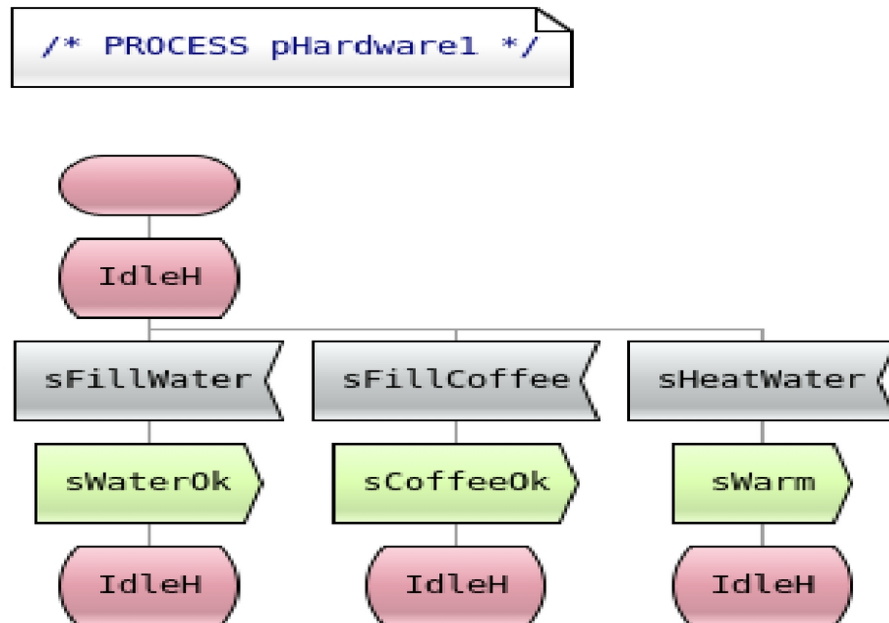
Remember that the environment is represented in SDL by a pseudo process (RTDS_Env) and thus it is not typically described by an EFSM.



Controller Process



Hardware Process



5. Implementing the MakeCoffee use case

In the accompanying archive, there is a compressed directory called `LinuxTut4code`. Once the archive is uncompressed change to that directory. Inside it there are three directories (`05-EFSM`; `06-Timer`; `07-EFSM_Timer`). Each one of them has a `Makefile`. Use `make` to compile the source code in each directory.

Inside the `05-EFSM` directory you can execute the `CoffeeMachineFSM` program or the `CoffeeMachineFSMtrc` program.

The `CoffeeMachine.c` file implements the `MakeCoffee` use case for the Coffee Machine model v1. It includes the `phtrdsMsgLyr.h` header file. This file, in turn, includes the `pMLusrConf.h` header file. The latter file should be modified by the developer since it defines the number of queues (`NUM_QUEUES`) you will use (one per each EFSM receiving process), the message structure (`msg_t`), the signals, and the states of the EFSMs in the model.

For this model we have two queues (`CONTROLLER_Q` and `HARDWARE_Q`; see `pMLusrConf.h`). The following table summarises queue usage:

Queue	queue [CONTROLLER_Q]	queue[HARDWARE_Q]
Owner Process	pController	pHardware

Since the model has two types of messages, namely pure messages and one impure message (`sCoin`), which carries one single parameter, the following C struct is appropriate to represent both types:

```
typedef struct
{
    int    signal;
    int    value;
} msg_t;
```

Signals are encoded using symbolic names in an enumerated type as shown in the following Table (the corresponding numeric encoding is shown between parenthesis):

To Controller	To Hardware
sCoin (0)	sFillWater (0)
sCoffee (1)	sFillCoffee (1)
sTea (2)	sHeatWater (2)
sWaterOK (3)	
sCoffeeOK (4)	
sWarm (5)	

States are encoded as follows:

pController	pHardware
IdleC (0)	IdleH (0)
PaidTen (1)	
PouringWater (2)	
DispensingCoffee (3)	
BrewingCoffee (4)	

Of course, you should change the definitions according to your design. Notice that it is necessary to use two different state names for the `Idle` states in process `pHardware` (`IdleH`) and `pController` (`IdleC`). This is because C does not allow using the same name more than once.

The `CoffeeMachine` Linux process has four threads. The initial thread, which executes the `main` function, initialises the message queuing layer and creates the SDL system (`pController` and `pHardware` SDL processes) plus the `RTDS_Env`. The SDL processes correspond to two of the remaining three threads: the first one, corresponds to the SDL `pController` process, and executes the `pController` function; the second one, corresponds to the SDL `pHardware` process, and executes the `pHardware` function. The third created thread emulates the `RTDS_Env` pseudo-process in `PragmaDev Studio` and executes the `pCustomer` function, and is in charge of interacting with the user.

The `pController` and `pHardware` functions represent actual SDL processes and therefore follow a common structure, which we will use to describe an EFSM using threads. The statements before entering the **for** loop are used to model the first transition in the EFSM (from the `START` symbol to the first state). Once inside the loop, the EFSM waits for an incoming message by using the following statement:

```
InMsg = receiveMessage ( &(amp;queue [Q_NUMBER]) );
```

where `Q_NUMBER` represents the queue number as defined in the `pMLusrConf.h` file: `CONTROLLER_Q` (for the SDL `pController` process) and `HARDWARE_Q` (for the SDL `pHardware` process).

Once a message is received, there is a **switch** statement to evaluate which is the current state (**switch** (`state`)) and for each state, there is another **switch** statement to evaluate the received signal (**switch** (`InMsg.signal`)) within that state. Once the signal in a given state is evaluated, the following statements describe the actions corresponding to the equivalent transition.

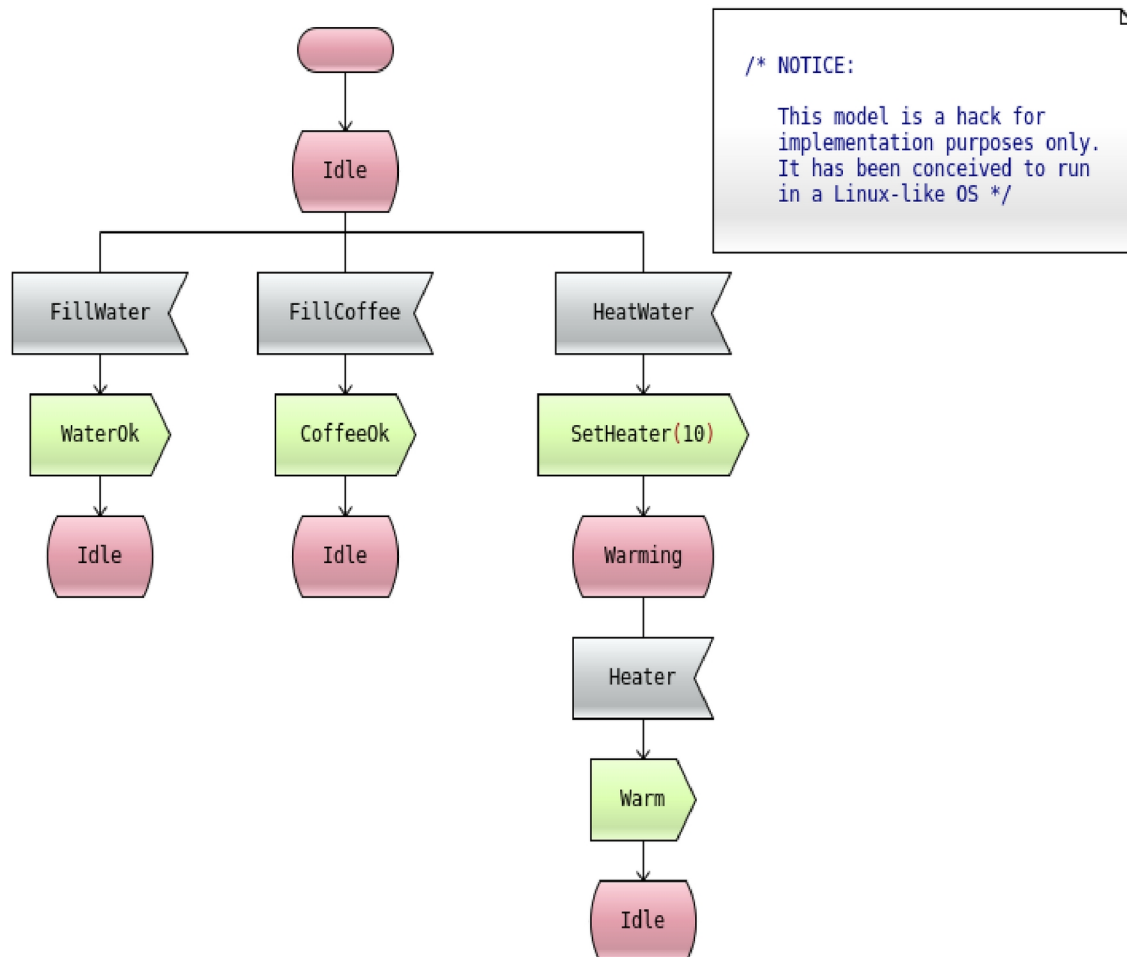
The `CoffeeMachine` program has no instrumentation to follow the execution; the `CoffeeMachineTrc` program prints some messages that indicate for each SDL process the received messages (signal name and value) in the current state as well as the next state. This approach allows following the evolution of the EFSMs via a textual “*MSC execution trace*” of the running program.

To do so, the `CoffeeMachineTrc.c` file just adds some `printf` statements as an aid to follow the evolution of the threads representing the SDL processes. Notice that in order to do so you will need to make use of the Tables that establish the equivalences between the enumerations defined in `pMLusrConf.h` for both signals and states and their corresponding numeric values. Since we are using a single terminal to display the actions of all of the three threads, the output of every thread starts at a different “column”: the left-hand side “column” is used by the `pCustomer` thread; the middle “column” is used by `pController` thread; the `pHardware` thread uses the right-hand side “column”.

Inside the 05-EFSM directory there is another directory called `UsingSymbolicNames`, that has an improved version of the `CoffeeMachineTrc.c` program. It contains a more elaborate scheme to follow the evolution of the threads. This time it uses symbolic constants (literal strings) to represent the states and the messages, so it is easier to follow the system evolution. Here we will use the C pre-processor to convert the symbolic constants to its equivalent literal strings.

6. Adding a Timer to the Coffee Machine

According to our second SDL model, the timer should expire after 10 time units and it is to be set in conjunction with the activity of heating the water as shown in the following SDL process diagram:



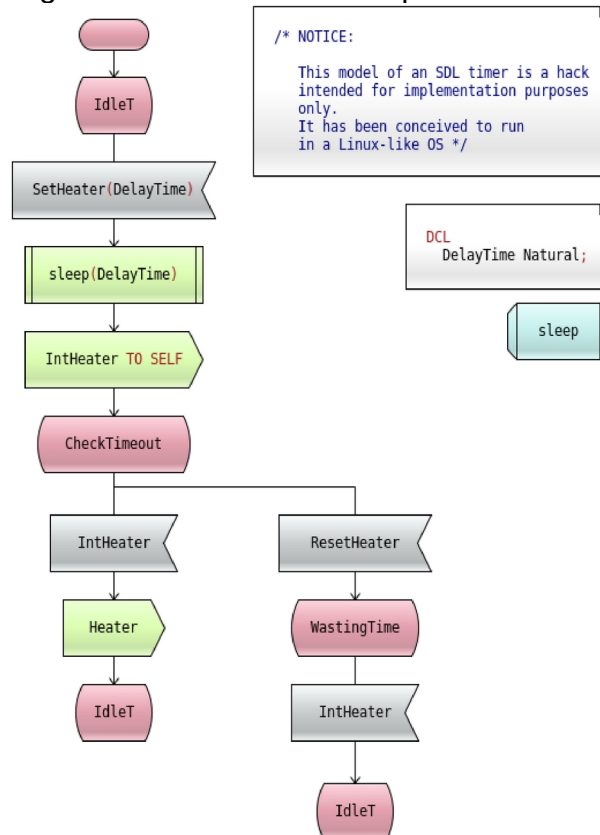
We need to implement SDL timers emulating them in Linux via an EFSM encoded in a Linux thread. To use the emulated timer, we need first to add some signals, as the following Table shows:



To Controller	To Hardware	To Timer
sCoin	sFillWater	sSetHeater
sCoffee	sFillCoffee	sTexpired ²
sTea	sHeatWater	sResetHeater
sWaterOK	sHeater	
sCoffeeOK		
sWarm		

The SET (NOW + 10, Heater) statement in SDL will be implemented in Linux via sending a message: sSetHeater (10). Likewise, the RESET (Heater) statement in SDL would be implemented as a message: sResetHeater. To model the timer itself in Linux, we must create a thread that makes use of the sleep function, which yields the processor for a specified number of seconds. Recall that the name of the signal sent when the timer expires is the name of the timer itself (Heater in this case).

The following SDL process diagram illustrates how to implement a timer in Linux:



- 2 The sTexpired signal as defined in the Linux code corresponds to the IntHeater signal in the SDL diagram below.

The corresponding example code is inside the `06-Timer` directory. It has two source code files: `CoffeeMachine.c` and `CoffeeMachineTrc.c`, plus the files that implement the messaging layer. These programs implement the `MakeCoffee` use case for the Coffee Machine model v2, and incorporate a `pTimer` function, whose purpose is to emulate the functionality of a SDL timer. Use `make` to compile the source code in that directory. As before, you can execute the `CoffeeMachineFSM` program or the `CoffeeMachineFSMtrc` program. In the latter case you will get an “execution trace” of the running program using symbolic names.

The code in the `pTimer` function corresponds to the previous SDL process diagram.

Last but not least, the `07-EFSM_Timer` directory has one source code file (`CoffeeMachineTrc.c`) plus the files that implement the messaging layer. This program implements the `MakeCoffee` use case for the Coffee Machine model v3. In this version, the hardware is described by three SDL processes (`pWaterD`, `pCoffeeD`, and `pHeater`) that require creating three additional threads, as well as a new per thread queue (`queue [WATERD_Q]`, `queue [COFFEED_Q]`, and `queue [HEATER_Q]`). This of course, implies modifying the `pMLusrConf.h` file, writing the corresponding functions (`pWaterD`, `pCoffeeD`, and `pHeater`), and modifying the `main` and `pController` functions.

7. Logging the execution output to a file

In UNIX (as well as in Linux) almost every communication with a peripheral is assimilated to a file stream, which means that processes can manipulate hardware peripherals by using the very same functions and operations destined to work with files. A notable exception is when you establish a network connection.

When a program is started, three file streams are automatically opened. They are `stdin`, `stdout`, and `stderr`. The first one is associated to standard input (the keyboard), while the second is associated to standard output (the screen). The last one is meant to be used for logging error messages (standard error). Having a dedicated stream for that purpose allows separating the normal output of the program, which typically goes to `stdout`, from error messages that may be desirable to log and that arise from anomalies during the program's execution.

Furthermore, using file streams to represent peripherals has many advantages. Amongst them, it is possible to redirect the output of one program to another file stream aside from `stdout`. This approach allows us to save the resulting output of a program's execution to a file (as well as to use it as input to another process.)

This feature can be used to illustrate the execution of our programs. To do so, you can redirect the output of a process to a given file by using the `|` operator and the `tee` command by following the syntax `prog1 | tee outfile`, where `prog1` is the name of the program whose output we want

to preserve, and `outfile` is the name of the file where we will store the output of `prog1`.

`tee` will take as its input the output of `prog1` and will write it to both `stdout` and `outfile`.

As a way of example, assuming that you current directory is `LinuxTut4code/07-EFSM_Timer`, you may enter the following line in front of the command prompt:

```
./CoffeeMachineFSMtrc | tee CM_trc.out
```

Please notice that if you enter `0` when asked for the coin value, the `exit` function will cause the Linux process (as well as its threads) to terminate.

Once you finish the execution, you can check the resulting execution output so far by entering:

```
cat CM_trc.out
```

As you can see, the contents of the `CM_trc.out` file is identical to that generated by the process. Besides, since it is in plain text, it can be pasted as part of a report for documentation purposes.

Be aware however that the `|` operator causes `outfile` to be obliterated. Should you need to preserve the contents of `outfile` and append to it some other stuff, you must use the `-a` option.

8. Next Steps

To compile your own use cases, use the `CoffeeMachine.c` as a template. Modify the `pMLusrConf.h` header file according to your needs: update the number of queues (`NUM_QUEUES`), one per each EFSM receiving process; define the manifest constants to define each message queue; the message structure (`msg_t`); the signals; and the states of the EFSMs in the model. Then modify the code to create the SDL system and add the required functions to be executed by the threads.